5 Reducers: 1 hour 50 minutes

10 Reducers: 2 hours 5 minutes

20 Reducers: 1 hour 55 minutes

The goal of this assignment was to implement the PageRank algorithm in Hadoop to determine the most popular pages from a dataset of Wikipedia pages. The implementation needed to work both locally (using a small example graph) and on Amazon EMR (using the provided large Wikipedia dataset). The Hadoop jobs implemented were init, iter, diff, join, and finish, each responsible for a specific step in the iterative PageRank process.

The provided codebase included the vast majority of the needed code, the only thing we needed to implement was some of the functionality for each mapper and reducer. In the init files, the mapper read each like of the input file and emitted an initial rank for each node along with its associated list. The reducer then aggregated adjacent lists and produced an output that was subsequently read by the iter job, which took a nodes rank and divided it evenly between its links, then split them up and emitted them as separate values. The diff job ran in two phases and compared the ranks from two consecutive iterations to figure out the largest absolute change in rank for any node. The program used this output to determine if convergence occurred. After that, the join job did a reduce side join between the ranks and the page names, producing readable pairs. The finish job was then responsible for sorting the output by rank in descending order.

Locally, I tested with a small file that had values ranging from 1-5 and each node had a varying number of links. I purposefully made 4 the most popular to show how the code was able to clearly display the breakdown. Once I was confident that the code worked for my local, much smaller test file, I switched over to emr and ran it on the full scale wikipedia files. I tested different numbers of reduces to see the difference, as noted above. With five reducers, the code finished in about one hour and fifty minutes, with ten it took two hours and five minutes, and with twenty reducers, it took

one hour and fifty-five minutes. The ranks themselves did not change much between runs, but the times made it clear that more reducers does not directly lead to faster execution.

This assignment was a good exercise in building a mapreduce pipeline, understanding how to keep intermediate data consistent across iterations, and seeing how changing things, like the number of reducers, can impact the end result.